

Projeto ToDo List: *a primeira versão*

Hora de juntar tudo num projeto. Nesta aula, o agente do Cursor constrói a **primeira versão** de uma lista de tarefas em **Python + Streamlit**: adicionar, listar e remover tarefas. Você prepara o ambiente, escreve uma rule, revisa o plano, dá o prompt, roda, testa no navegador e faz o primeiro commit.

Python + Streamlit

Rule do projeto

Plan → Agent

streamlit run

Teste no Browser

Primeiro commit

Formação Claude Code 2026 · Baseado em cursor.com/docs (Agent, Plan Mode, Rules, Browser) e docs.streamlit.io, consultados em outubro de 2026 (Cursor 3.22, Streamlit 1.65) · Código de referência testado em `projeto/app.py` · Tela do app: print real

Um app, *três aulas*

AULA 10 • HOJE

Primeira versão: adicionar, listar e remover tarefas



AULA 11

Marcar tarefas como concluídas



AULA 12

Salvar as tarefas num banco de dados

POR QUE PYTHON + STREAMLIT

O **Streamlit** transforma um script Python num app web, sem HTML, CSS ou JavaScript. Um arquivo só já tem tela, campos e botões, e roda no navegador em `localhost`. É ótimo para ver o agente trabalhar sem se perder em configuração.

O QUE VOCÊ VAI PRATICAR

Ask para entender, **Plan** para revisar a abordagem, **Agent** para construir, **rules** para padronizar, **Browser** para testar (aula 09) e **checkpoints** e Git para voltar atrás (aula 01).

Prefere outra tecnologia (HTML/JS, React, Flask)? O processo desta aula é o mesmo; só troque a stack no prompt e na rule.

O que entra *na versão 1*

ENTRA

- ✓ Campo de texto e botão **Adicionar**
- ✓ Lista das tarefas com botão **Remover** em cada uma
- ✓ Aviso ao tentar adicionar tarefa **vazia**
- ✓ Mensagem quando a lista estiver vazia e contador de tarefas
- ✓ Textos da interface em **português**

FICA PARA DEPOIS

- Marcar como concluída (aula 11)
- Guardar as tarefas depois de recarregar a página (aula 12)
- Login, prazos, categorias, edição

Escopo escrito antes evita o agente "inventar" funcionalidades. Pedidos pequenos e verificáveis dão resultados melhores do que "faça um app completo".

Preparando *o ambiente*

PASSO	WINDOWS (POWERSHELL)	LINUX E MACOS
Pasta do projeto	<code>mkdir todo-app; cd todo-app</code>	<code>mkdir todo-app && cd todo-app</code>
Ambiente virtual	<code>py -m venv .venv</code>	<code>python3 -m venv .venv</code>
Ativar	<code>.venv\Scripts\Activate.ps1</code>	<code>source .venv/bin/activate</code>
Instalar	<code>pip install streamlit</code>	
Git	<code>git init</code>	
Abrir no Cursor	<code>cursor .</code> (ou File > Open Folder)	

No Debian/Ubuntu, se o `venv` reclamar de `ensurepip`, instale o pacote `python3-venv` (por exemplo, `sudo apt install python3-venv`).

Quer delegar? Peça ao agente para fazer esses passos: ele roda os comandos no terminal, com a sua aprovação (Run Modes).

Uma rule *para o projeto*

.CURSOR/RULES/PROJETO.MDC

```
---
description: Padrões do projeto ToDo List
alwaysApply: true
---
- Projeto em Python 3 com Streamlit, num único
  arquivo app.py por enquanto.
- Textos da interface e mensagens em português
  do Brasil.
- Código simples e legível, com nomes em
  português; comentários só onde ajudam.
- Não adicione bibliotecas além do Streamlit
  sem perguntar.
- Mantenha o requirements.txt atualizado.
```

POR QUE UMA RULE

Ela entra em **todas** as conversas do projeto, em qualquer modo. Assim você não repete "em português", "sem bibliotecas novas" a cada pedido, e as aulas 11 e 12 seguem o mesmo padrão.

Com `alwaysApply: true`, a rule vale sempre. Rules com `globs` valem só para certos arquivos (módulo 35).

Também dá para pedir ao agente: *"crie uma rule do projeto com estes padrões"*.

Primeiro *o plano*

Mude para o modo **Plan** (`Shift` + `Tab`) e peça a abordagem antes do código. Você revisa e só depois o agente edita.

Planeje a primeira versão de um app de lista de tarefas em Streamlit, num arquivo `app.py`.
Funcionalidades: adicionar tarefa (campo + botão), listar as tarefas com um botão Remover em cada uma, avisar se a tarefa estiver vazia, mostrar uma mensagem quando não houver tarefas e um contador. Sem banco de dados: as tarefas ficam em `st.session_state`. Não escreva código ainda.

O QUE CONFERIR NO PLANO

- ✓ Usa `st.session_state` para guardar a lista
- ✓ Usa um formulário que limpa o campo depois de adicionar
- ✓ Remover usa uma chave única por botão
- ✓ Não inventou banco, login nem bibliotecas extras
- ✓ Cria `requirements.txt`

Discorda de algo? Responda no chat e peça um plano ajustado. Quando estiver bom, aprove e deixe o agente executar.

O prompt *de construção*

Se preferir ir direto no modo **Agent**, um prompt completo faz o mesmo papel. Ele diz o que construir, o que não fazer e como verificar:

```
Crie a primeira versão de um app de lista de tarefas em Streamlit, no arquivo app.py:
- título "Minhas tarefas";
- formulário com o campo "Nova tarefa" e o botão "Adicionar", que limpa o campo ao enviar;
- avise com st.warning se a tarefa for vazia;
- liste as tarefas, cada uma com um botão "Remover", e mostre quantas são;
- sem tarefas, mostre uma mensagem com st.info.
Guarde a lista em st.session_state (sem banco).
Crie o requirements.txt. No fim, rode o app e me diga a URL.
```

PARTE DO PROMPT	FUNÇÃO
Arquivo e stack	Diz onde e com quê
Lista de comportamentos	Vira o critério de "pronto"
"Sem banco"	Segura o escopo da v1
"Rode e me diga a URL"	Pede verificação, não só código

Ditar (aula 02) funciona bem aqui: fale a lista de comportamentos e revise os nomes técnicos (`st.session_state` , `app.py`) antes de enviar.

O código *da versão 1*

```
import streamlit as st

st.set_page_config(page_title="ToDo List", page_icon="✅")
st.title("✅ Minhas tarefas")

# A lista vive na sessão do navegador: some ao recarregar
# a página (v1, sem banco).
if "tarefas" not in st.session_state:
    st.session_state.tarefas = []

with st.form("nova_tarefa", clear_on_submit=True):
    col_texto, col_botao = st.columns([4, 1], vertical_alignment="bottom")
    texto = col_texto.text_input("Nova tarefa",
                                placeholder="O que você precisa fazer?")
    adicionar = col_botao.form_submit_button("Adicionar",
                                              width="stretch")

if adicionar:
    texto = texto.strip()
    if not texto:
        st.warning("Digite uma tarefa antes de adicionar.")
    else:
        st.session_state.tarefas.append(texto)

if not st.session_state.tarefas:
    st.info("Nenhuma tarefa ainda. Adicione a primeira acima.")
else:
    st.caption(f"{len(st.session_state.tarefas)} tarefa(s)")
    for i, tarefa in enumerate(st.session_state.tarefas):
        col_tarefa, col_remover = st.columns([5, 1],
                                              vertical_alignment="center")

        col_tarefa.write(tarefa)
        if col_remover.button("Remover", key=f"remover_{i}"):
            st.session_state.tarefas.pop(i)
            st.rerun()
```

COMO LER

- **session_state:** o Streamlit roda o script de novo a cada clique; o `session_state` guarda a lista entre essas execuções
- **form + clear_on_submit:** envia o campo e o limpa em seguida
- **strip():** trata " " como vazio
- **key única:** cada botão Remover precisa de uma chave própria
- **st.rerun():** redesenha a lista logo após remover

Código de referência na pasta da aula: `projeto/app.py` e `projeto/requirements.txt`. O do seu agente pode sair diferente, e tudo bem, desde que cumpra o escopo.

Rodando *o app*

TERMINAL DO CURSOR (COM A .VENV ATIVADA)

```
streamlit run app.py
```

```
You can now view your Streamlit app in your browser.  
Local URL: http://localhost:8501
```

- ✓ A porta padrão do Streamlit é a **8501**
- ✓ Ao salvar `app.py`, a página oferece **Rerun** para atualizar
- ✓ `Ctrl` + `C` no terminal para parar o servidor

DEIXE O AGENTE RODAR

Se você pediu "rode o app", o agente executa o comando no terminal (com aprovação) e devolve a URL. Ele também percebe quando o servidor **já está rodando** e reaproveita a porta em vez de abrir outro.

"streamlit: command not found": a `.venv` não está ativada no terminal, ou o Streamlit não foi instalado nela. Ative e rode `pip install -r requirements.txt`.

A versão 1 *funcionando*

Deploy

✓

Minhas tarefas

Nova tarefa

O que você precisa fazer?

Adicionar

3 tarefa(s)

Estudar a aula de MCP

Remover

Revisar o PR #22

Remover

Comprar café

Remover

Print real do `projeto/app.py` rodando no Streamlit 1.65, com três tarefas de exemplo

Testando *com o agente*

TESTE PELO NAVEGADOR (AULA 09)

@Browser Abra o app em localhost:8501 e teste:
1. adicionar "Estudar MCP" e "Revisar PR";
2. clicar em Adicionar com o campo vazio;
3. remover "Estudar MCP".
Para cada passo, diga o que esperava e o que aconteceu. Não altere código.

Peça o relatório **antes** de pedir correções: assim você decide o que mudar.

TESTE	ESPERADO
Adicionar duas tarefas	Aparecem na ordem, contador "2 tarefa(s)", campo limpo
Adicionar vazio ou só espaços	Aviso "Digite uma tarefa antes de adicionar."
Remover uma tarefa	Some da lista na hora; contador "1 tarefa(s)"
Remover a última	Mensagem "Nenhuma tarefa ainda..."
Recarregar a página (F5)	A lista zera: esperado na v1, sem banco

O código de referência passou nesses testes com o `AppTest` do Streamlit (`streamlit.testing.v1`), que simula cliques sem abrir o navegador. Peça ao agente: *"crie testes com AppTest para estes casos"*.

Ajustes *e pontos de volta*

PEDIDOS PEQUENOS DE AJUSTE

Não deixe adicionar uma tarefa repetida; avise com `st.warning`.

Limite o texto da tarefa a 100 caracteres e mostre o contador de caracteres no campo.

Se eu clicar em Remover sem querer, quero poder desfazer a última remoção.

- ✓ Um ajuste por mensagem; teste antes do próximo
- ✓ Não gostou? **Restore Checkpoint** na mensagem anterior
- ✓ Mudou de assunto? Conversa nova (`Ctrl` + `N`)
- ✓ Ficou bom? **Commit** antes do próximo ajuste

Cuidado com o escopo: "concluídas" e "salvar no banco" são das próximas aulas. Guarde a vontade.

Checklist *e primeiro commit*

- ✓ Adiciona, lista e remove tarefas
- ✓ Rejeita tarefa vazia com aviso
- ✓ Mostra a mensagem de lista vazia e o contador
- ✓ Textos em português
- ✓ `requirements.txt` com `streamlit`
- ✓ `.venv` fora do Git

TERMINAL

```
# .gitignore
.venv/
__pycache__/

git add .
git commit -m "ToDo List: primeira versão (adicionar, listar,
remover)"
```

Também dá para pedir ao agente: *"crie o .gitignore para Python e faça o commit da primeira versão"*. Leia a mensagem de commit antes de aprovar.

Com o MCP do GitHub (aula 08), o agente ainda pode criar o repositório remoto e subir o código.

Teste o que *ficou*

1. Onde a v1 guarda as tarefas?

A) Num banco SQLite

B) Em st.session_state

C) Num arquivo .txt

D) No GitHub

B. Por isso somem ao recarregar a página.

2. Para que serve a rule `.cursor/rules/projeto.mdc` com `alwaysApply: true` ?

A) Instalar o Streamlit

B) Aplicar os padrões do projeto em todas as conversas

C) Rodar o app

D) Bloquear o agente

B. Sem repetir as regras a cada pedido.

3. Qual comando roda o app?

A) python app.py

B) streamlit run app.py

C) npm start

D) cursor run

B. Abre em localhost:8501.

4. Por que pedir um plano no modo Plan antes do código?

A) É obrigatório

B) Para revisar a abordagem antes de qualquer edição

C) Fica mais barato sempre

D) O Agent não sabe criar arquivos

B. Você corrige o rumo antes do código existir.

A primeira versão *está no ar*

Com escopo definido, uma **rule** do projeto, um **plano** revisado e um prompt com critério de pronto, o agente construiu o ToDo List em **Streamlit**: adicionar, listar e remover. Você rodou, testou pelo **Browser** e fez o primeiro **commit**. Na próxima aula, as tarefas ganham o estado de concluídas.

Escopo antes

Rule · Plan · Agent

`st.session_state`

`streamlit run app.py`

Testar · commitar

Próxima aula → 11 · Projeto ToDo List · Adicionando Tarefas Concluídas

Fontes: cursor.com/docs (Agent Overview, Plan Mode, Rules, Browser); docs.streamlit.io (`st.session_state`, `st.form`, `st.columns`, `st.rerun`, `AppTest`), consultados em outubro de 2026. Código de referência em `projeto/app.py`, testado com Streamlit 1.65; a tela do app é um print real.